

Kernkonzepte von Databricks Apps

Die wichtigsten Grundbegriffe: Was eine App technisch ist, wie sie Ressourcen nutzt, Zustand speichert und Nutzer authentifiziert.

Diese Seite erklärt die zentralen Begriffe, die man verstehen sollte, bevor man eigene Apps entwickelt. Sie bildet die Grundlage für alle weiteren Themen wie Konfiguration, Deployment und Betrieb.

Was ist eine Databricks App technisch gesehen?

Eine Databricks App ist eine Webanwendung, die als **containerisierter Dienst** auf der serverlosen Databricks-Plattform läuft. "Containerisiert" bedeutet: Die App läuft isoliert in einer Art abgeschlossener Box (Container), mit eigener Konfiguration, eigener Identität und eigener Laufzeitumgebung.

Jede App gehört zu einem bestimmten Workspace und kann sowohl auf Workspace-Ressourcen (z. B. SQL-Warehouses) als auch auf Account-weite Ressourcen (z. B. Unity Catalog) zugreifen. Optional kann man eine App auch mit Nutzern außerhalb des eigenen Workspace teilen, solange diese im selben Account sind.

Technisch betrachtet läuft die App als Dienst in der "Control Plane" (dem Verwaltungsbereich von Databricks) und stellt dort die Weboberfläche bereit, kann aber auf Dienste in der "Data Plane" (dem Bereich, in dem die eigentlichen Daten verarbeitet werden) zugreifen – sowohl serverlose als auch klassische Ressourcen. Verwaltet werden Apps über den Bereich "Apps" in der Workspace-Oberfläche.

Architektur und Isolation

Apps laufen auf einer serverlosen Compute-Architektur mit mehreren Schutzschichten:

- Dedizierte (also exklusiv zugewiesene) Rechenressourcen
- Netzwerksegmentierung (Trennung des Datenverkehrs)
- Verschlüsselung von Daten – sowohl gespeichert als auch während der Übertragung
- Prinzip der minimalen Rechtevergabe ("Least Privilege")

Die App-URL

Databricks generiert automatisch eine eindeutige URL nach folgendem Muster:

```
https://<app-name>-<workspace-id>.<region>.databricksapps.com
```

Die Bestandteile sind der gewählte App-Name, die Workspace-ID und die Cloud-Region. Diese URL kann nachträglich nicht geändert werden – wer eine andere URL braucht, muss eine neue App-Instanz erstellen.

Vorlagen (Templates)

App-Vorlagen sind vorgefertigte Grundgerüste, die den Entwicklungsstart beschleunigen. Jede Vorlage enthält typischerweise:

- Eine grundlegende Dateistruktur
- Eine `app.yaml` -Datei, die den Startbefehl definiert (z. B. `streamlit run <app-name>`)
- Konfiguration von Umgebungsvariablen
- Deklaration benötigter Ressourcen
- Eine `requirements.txt` (Python-Pakete) bzw. `pyproject.toml` (bei Nutzung von uv), oder `package.json` für Node.js-Pakete

Vorlagen lassen sich über die Databricks-Oberfläche oder die Kommandozeile (CLI) erzeugen.

Systemumgebung

Databricks Apps laufen in einer von Databricks vorkonfigurierten und verwalteten Systemumgebung. Jede App hat eine isolierte Umgebung, damit sich Abhängigkeiten verschiedener Apps nicht gegenseitig stören. Benötigte Pakete und Versionen müssen explizit angegeben werden:

- Python: in `requirements.txt` oder `pyproject.toml`
- Node.js: in `package.json`

Beim Deployment installiert Databricks die deklarierten Abhängigkeiten in einer isolierten Laufzeitumgebung. Falls ein Paket bereits vorinstalliert ist, überschreibt die angegebene Version den Standard.

App-Ressourcen

Ressourcen sind Databricks-native Dienste, von denen eine App abhängt. Sie werden in der Datei `databricks.yml` im Feld `resources` deklariert. Unterstützte Ressourcentypen sind:

- SQL-Warehouse
- Job (geplanter Auftrag)
- Model-Serving-Endpunkt (für KI-Modelle)
- Genie-Agent
- Secret (geschütztes Zugangsdatum)
- Volume (Dateispeicher)

Für Dienste, die nicht direkt unterstützt werden, können über Unity-Catalog-verwaltete Secrets Zugangsdaten sicher eingebunden werden.

Zwei-Phasen-Modell: Deklaration und Konfiguration

Die Einrichtung von Ressourcen erfolgt in zwei Schritten:

1. **Deklaration (bei der Entwicklung):** Im Manifest `databricks.yml` wird festgelegt, welche Ressourcen und Berechtigungen die App benötigt.
2. **Konfiguration (beim Deployment):** Administratoren verknüpfen die deklarierten Ressourcen mit konkreten, workspace-spezifischen Instanzen über die Apps-Oberfläche (z. B. Auswahl eines bestimmten SQL-Warehouses).

Diese Trennung macht Apps portabel: Derselbe Code kann unverändert in Entwicklungs- und Produktivumgebung eingesetzt werden, während er jeweils mit den passenden, umgebungsspezifischen Ressourcen verbunden wird. Man sollte daher **keine Ressourcen-IDs oder umgebungsspezifischen Werte fest im Code verankern** (Hardcoding vermeiden).

Es gilt das Prinzip der minimalen Rechte: Apps können nur bestehende Ressourcen nutzen, aber keine neuen erstellen. Workspace-Administratoren prüfen und genehmigen angeforderte Ressourcenzugriffe beim Deployment und vergeben die nötigen Berechtigungen an das Service Principal (die technische Identität der App).

App-Status

Eine App kann sich in einem von vier Zuständen befinden:

Status	Bedeutung
Running (Läuft)	Aktiv und erreichbar; es fallen Kosten für die Rechenzeit an
Stopped (Gestoppt)	Nicht erreichbar und kostenlos; Konfiguration und Umgebung bleiben für einen Neustart erhalten
Deploying (Wird bereitgestellt)	Startet gerade; noch nicht erreichbar, noch keine Kosten
Crashed (Abgestürzt)	Start fehlgeschlagen oder unerwartet gestoppt; nicht erreichbar, keine Kosten; Logs stehen zur Fehlersuche bereit

App-Zustand (State)

Der "Zustand" umfasst alle Daten oder Kontextinformationen, die eine App über verschiedene Nutzersitzungen oder Interaktionen hinweg speichern muss. Daten, die nur im Arbeitsspeicher liegen, gehen bei einem Neustart verloren. Für dauerhafte Speicherung gibt es mehrere Optionen:

Speicherart	Eigenschaft
Arbeitsspeicher (In-Memory)	Temporär, nur innerhalb einer Sitzung; geht bei Neustart verloren
Lokales Dateisystem	Temporäre Dateien während der Ausführung; geht bei Neustart verloren
Databricks-SQL-Tabellen	Dauerhaft, für strukturierte Analysedaten
Workspace-Dateien	Dauerhaft, für unstrukturierte Daten
Unity-Catalog-Volumes	Dauerhaft, für unstrukturierte Daten mit Governance (Zugriffskontrolle)
Lakebase-Datenbankinstanzen	Dauerhaft, relationale, PostgreSQL-kompatible Datenbank

Typische Anwendungsfälle sind das Zwischenspeichern (Caching) von Abfrageergebnissen, das Speichern von Nutzereinstellungen oder das Protokollieren von Nutzeraktionen über mehrere Sitzungen hinweg.

App-Autorisierung

Databricks Apps nutzen **OAuth 2.0** für Authentifizierung (Wer bin ich?) und Zugriffskontrolle (Was darf ich?). Jede App besitzt zwei sich ergänzende Identitäten:

App-Autorisierung (Service Principal)

Databricks erstellt automatisch für jede App ein sogenanntes Service Principal. Dieses fungiert als technische Identität der App und erhält vom Entwickler bestimmte Berechtigungen. Alle Nutzer der App teilen sich diese Identität und haben somit Zugriff auf dieselben Berechtigungen. Dieses Modell eignet sich für Vorgänge, die unabhängig vom einzelnen Nutzer sind, z. B. System-Logging.

Nutzer-Autorisierung (User Authorization)

Hierbei wird die Identität des jeweiligen App-Nutzers für Authentifizierung und Zugriffskontrolle verwendet. Nutzer müssen zum Databricks-Account gehören, in dem die App bereitgestellt ist. Nach dem Single-Sign-On (einmaliges Anmelden) kann die App die Zugangsdaten des Nutzers verwenden, um auf verwaltete Ressourcen wie SQL-Warehouses zuzugreifen – unter Beachtung der feingranularen Unity-Catalog-Berechtigungen, ohne dem Service Principal selbst Zugriff zu geben.

Apps fordern im Manifest bestimmte OAuth-Berechtigungsbereiche (Scopes) an, die den Zugriff auf APIs und Ressourcen steuern – das unterstützt eine feingranulare, unternehmenstaugliche Zugriffskontrolle.

App-Nutzer

Nach dem Deployment teilt man Apps mit anderen, indem man ihnen die Berechtigung `CAN_USE` oder `CAN_MANAGE` auf die App-Instanz gibt. Nutzer müssen nicht zwingend im selben Workspace sein, aber Mitglied desselben Accounts. Für externe Nutzer ist eine Synchronisierung über den Identitätsanbieter mittels SCIM-Protokoll nötig.

Über CI/CD-Pipelines und Infrastructure-as-Code lassen sich identische Apps über Entwicklungs-, Staging- und Produktivumgebungen hinweg verteilen. Die zentrale Apps-Oberfläche erleichtert es Nutzern, verfügbare Apps zu finden und zu starten.

Die wichtigsten Punkte im Überblick

- Apps sind isolierte, containerisierte Dienste mit eigener Identität und eigener URL.

- Ressourcen werden im Manifest deklariert, aber erst beim Deployment konfiguriert – das macht Apps portabel.
- Dauerhafte Datenspeicherung muss aktiv über Databricks-Dienste umgesetzt werden.
- Es gibt zwei Autorisierungsmodelle: auf App-Ebene (Service Principal) und auf Nutzer-Ebene.
- Mechanismen zum Teilen ermöglichen die Verteilung über Workspace-Grenzen hinweg innerhalb eines Accounts.

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/key-concepts>

Stand: 2026-08-06